UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE AND TECHNOLOGY
SCHOOL OF ICT
COMPUTER AND SOFTWARE ENGINEERING DEPARTMENT
NYARUGENGE CAMPUS

MODULE CODE/TITLE: WEB DESIGN

Project Technical Report – Web Technology
TECHNICAL ASSESSMENT 2026

Project Title: Land Track — Smart Land Management System

Registration Number: **225058410**

Academic Year: **2025-2026**

Date of Submission: **19 July 2026**

Table of Contents

Table of Figures

List of Abbreviations

- AJAX — Asynchronous JavaScript and XML
- API — Application Programming Interface
- CRUD — Create, Read, Update, Delete
- CSS — Cascading Style Sheets
- ERD — Entity Relationship Diagram
- HTML — HyperText Markup Language
- JS — JavaScript
- JSON — JavaScript Object Notation
- UI — User Interface
- URL — Uniform Resource Locator

Chapter 1. Introduction

1.1. Historical Background of the Case Study

Land administration has traditionally relied on paper-based records kept in physical registries at district or sector offices. Ownership details, parcel boundaries, and transaction history were recorded manually in ledgers or loose files. This approach is slow, difficult to search, and vulnerable to damage, loss, or unauthorized alteration. In recent years, many land administration bodies have begun shifting toward digital systems that make it easier to register parcels, track ownership, and process applications for registration, transfer, or subdivision. This project, **LandTrack**, is inspired by that shift: it models a simplified digital front-end for land parcel registration and application tracking, focusing on demonstrating how a browser-based interface can organize this kind of civic data even without a full backend system in place.

1.2. Problem Statement

Manual, paper-based land registration processes are slow and error-prone. Applicants often have no way to check the status of a submitted application except by physically visiting an office. Records stored only on paper are difficult to search, easy to misplace, and offer no straightforward way to cross-reference an owner with all the parcels they hold, or a parcel with its full application history. There is a need for a simple, accessible interface where applicants can submit land registration requests and track their status, while administrative staff can review, organize, and act on those requests from a central dashboard.

1.3. Objectives of the Project

General objective: To design and implement a web-based interface for managing land parcel registration and tracking application status, without requiring a backend server or database engine.

- Allow an applicant to submit a land registration application together with owner and parcel details.
- Allow an applicant to track the status of a submitted application using a unique application ID.
- Allow an administrator to log in securely and manage owner, location, and parcel records.
- Allow an administrator to review submitted applications and update their status (submitted, under review, approved, rejected).
- Demonstrate a clean separation between a public-facing zone and an authenticated administrative zone.

1.4. Expected Outcomes

- A responsive, multi-page website covering both the public and administrative zones.
- A working application submission flow that generates a trackable application ID.
- A working status-tracking page that reflects an application's current state.
- An administrative dashboard with basic statistics and full management pages for owners, locations, parcels, applications, and admin accounts.
- Clear internal documentation (this report) explaining the design, structure, and limitations of the system.

1.5. Project Rationale (Importance to the Community)

A system like LandTrack, even as a simplified academic prototype, illustrates real value for a community: faster, more transparent land registration reduces the burden on both applicants and administrative staff, cuts down on lost paperwork, and gives applicants visibility into where their request stands without needing to travel to an office repeatedly. For students, building this project also reinforces practical skills directly relevant to real civic and government digitization efforts happening in many countries, including structuring data models, designing multi-role interfaces, and reasoning about the trade-offs of a frontend-only implementation.

Chapter 2. System Analysis and Design

2.1. System Analysis

2.1.1. Intended Users of the Project

- Applicants / Land Owners — members of the public who want to register a parcel of land or check the status of a previously submitted application. They do not need an account.
- Administrators — staff who log in to review applications, manage owner/location/parcel records, and update application status. Two roles are modelled: admin and super_admin.

2.1.2. Intended System Partners

In a full deployment, this kind of system would typically partner with a local government land office or a national land administration authority responsible for maintaining the authoritative parcel registry, as well as any office issuing title deeds, since the parcel record includes a title_deed reference. To be more specific I will have partnership with **Minifra.**

2.2. System Design

1. High-Level Architecture

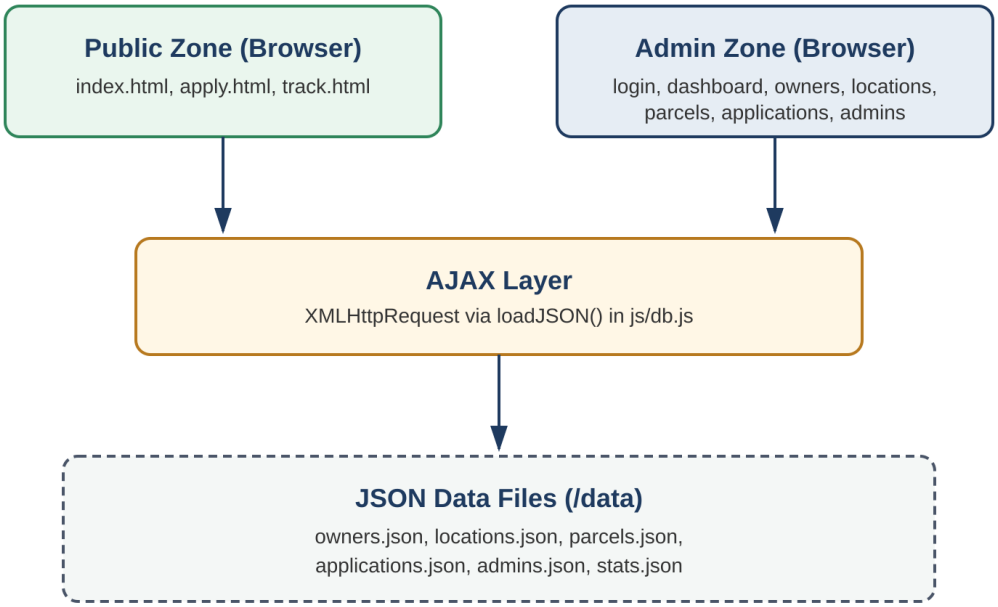


Figure 1: High-Level System Architecture

Since the project is restricted to HTML, CSS, and JavaScript with no backend server or database engine, data is simulated using JSON files under a /data folder. Every page loads the JSON it needs using AJAX (XMLHttpRequest, wrapped in a single loadJSON() helper function in js/db.js), then works with that data as plain in-memory JavaScript arrays. The site is split into a Public Zone, reachable without logging in, and an Admin Zone, which requires a login check on every page.

2. UI Design (Sketch of the UI Module) and Flowcharts

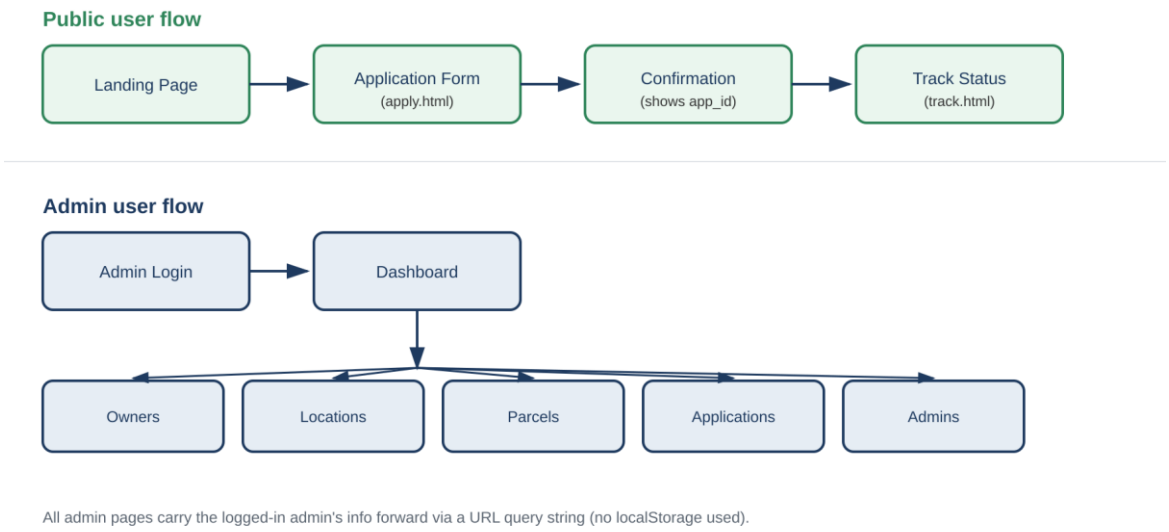


Figure 2: UI Navigation Flowchart — Public and Admin User Flow

The public flow takes a visitor from the landing page, through the application form, to a confirmation screen showing their application ID, and finally to the tracking page. The admin flow starts at login, lands on a dashboard, and fans out to five management pages (Owners, Locations, Parcels, Applications, Admins). Since the project does not use localStorage or sessionStorage, the logged-in admin's identity is carried between pages through the page URL's query string rather than through a stored session.

3. Database Design with ERD and Relationships

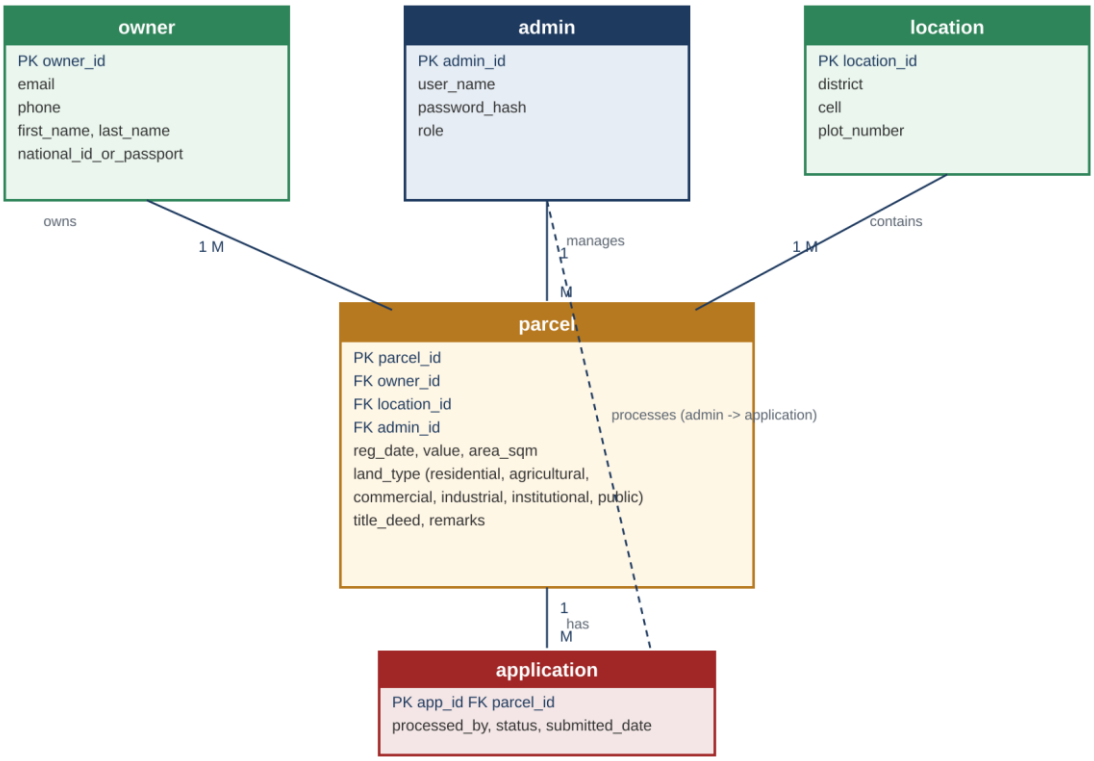


Figure 3: Entity Relationship Diagram (ERD)

The design has five entities: owner, location, parcel, application, and admin. An owner can own many parcels (1-to-many). A location can contain many parcels (1-to-many). An admin manages many parcels and processes

many applications (1-to-many in both cases). A parcel can have many applications submitted against it over time (1-to-many). Since there is no real database engine, these relationships are enforced by the interface itself — for example, the parcel registration form only lets an admin pick an owner and a location that already exist, rather than typing them freely.

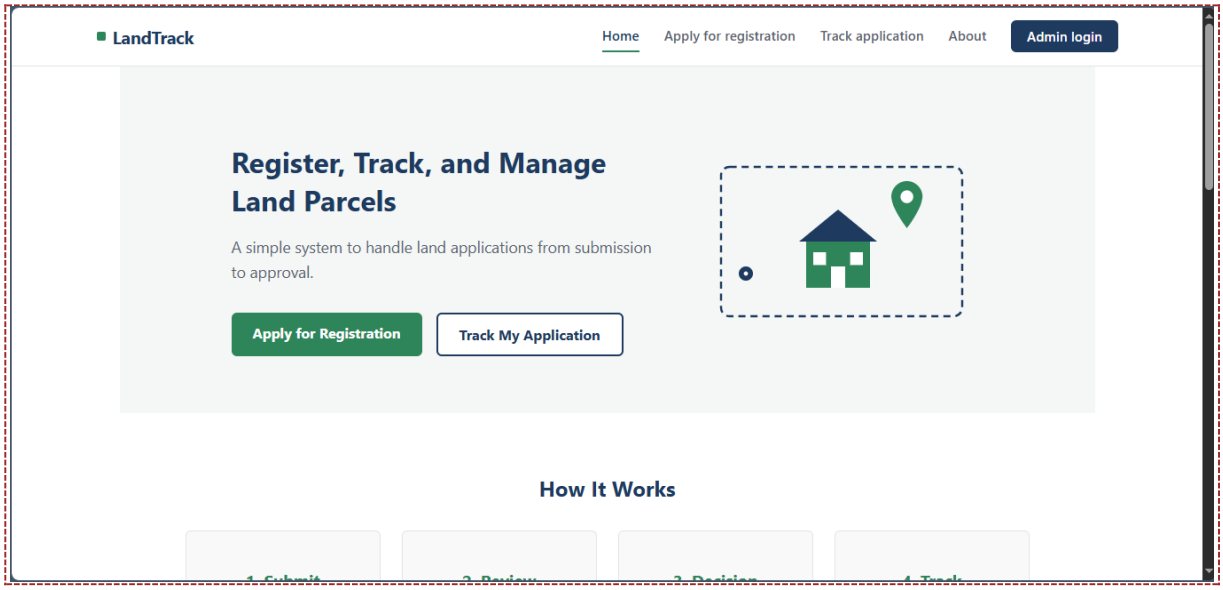
Chapter 3. Implementation

3.1. Introduction of the Section

This chapter presents the implemented pages of LandTrack, organized by zone. For each page, a screenshot should be inserted showing the page running in a browser (served through a local server, since the project uses AJAX to read local JSON files), followed by a short discussion of its purpose, how it works internally, and which files it depends on. The placeholder boxes below mark exactly where each screenshot needs to be inserted — they must be replaced with real screenshots taken from your own working copy of the project before submission.

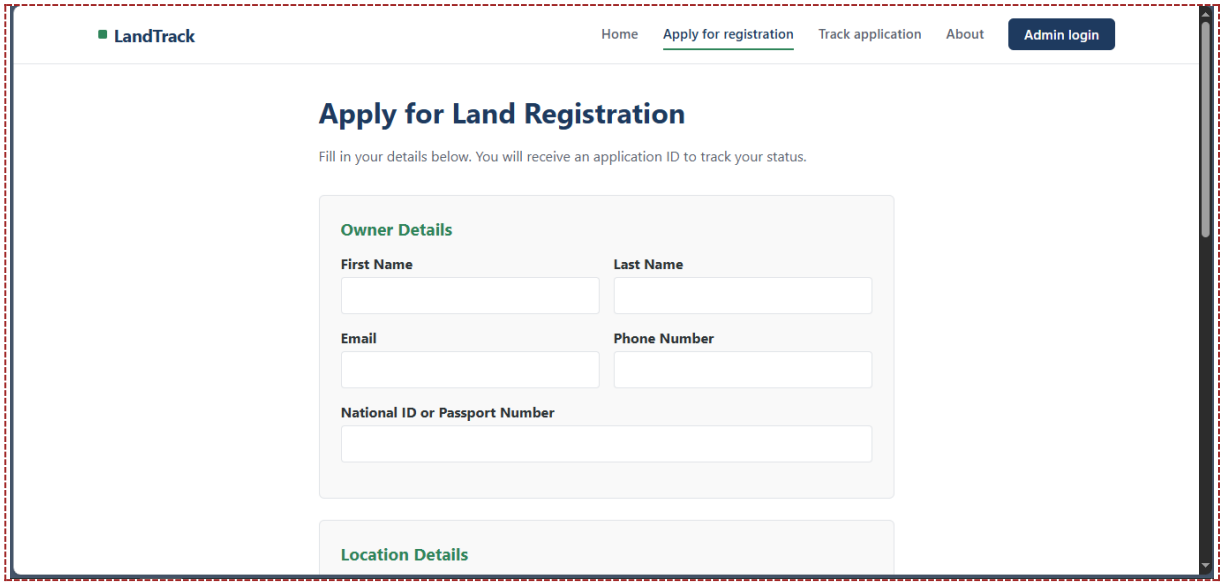
3.2. Screenshots and Discussion

Landing Page (index.html)



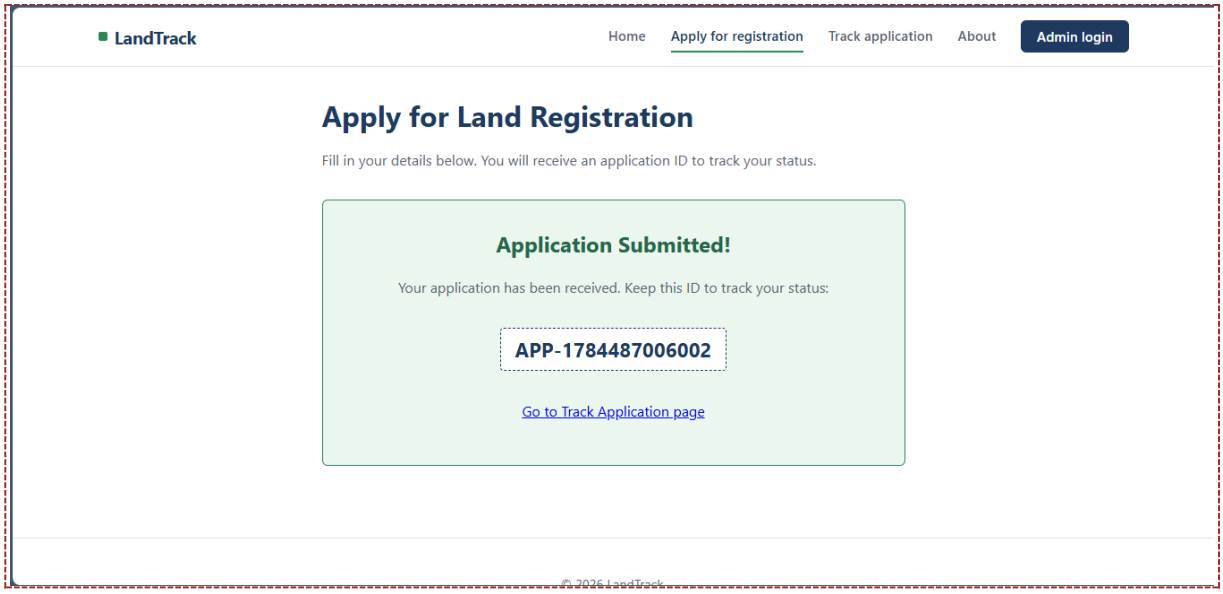
The landing page introduces the system with a hero section, a four-step "How It Works" explanation, a feature grid with icons, a live stats strip, and an about section. The stats strip is the only dynamic part of this page: js/index.js loads data/stats.json using AJAX (XMLHttpRequest) and fills in the parcelsRegistered, districtsCovered, and applicationsProcessed values. All navigation links (Home, Apply, Track, Admin Login) are handled by the shared navbar component, which also includes a responsive hamburger menu for small screens, implemented in js/navbar.js.

Application Form (apply.html)



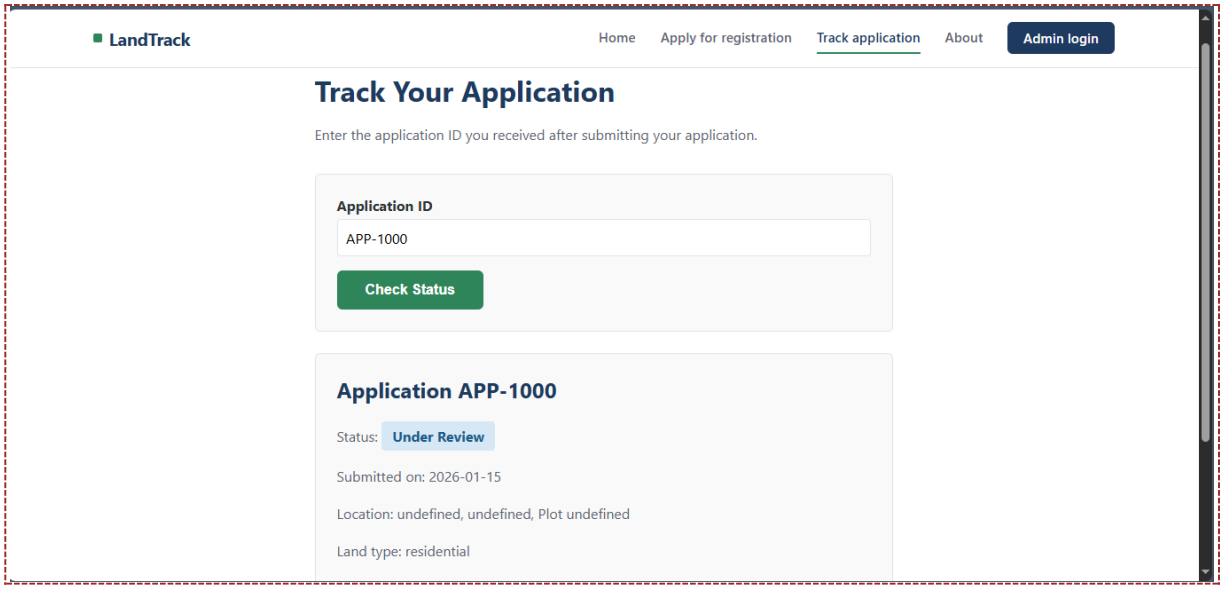
This page collects owner details, location details, and parcel details in a single form, grouped visually into three sections. On submission, js/apply.js first loads the existing owners.json (via AJAX) to check whether the typed national ID already belongs to a registered owner — reusing that record if so, or creating a new one if not. It then creates new location, parcel, and application records in memory, linking them by generated IDs, and displays the newly generated application ID in a confirmation box so the applicant can use it to track their status later.

Confirmation Screen



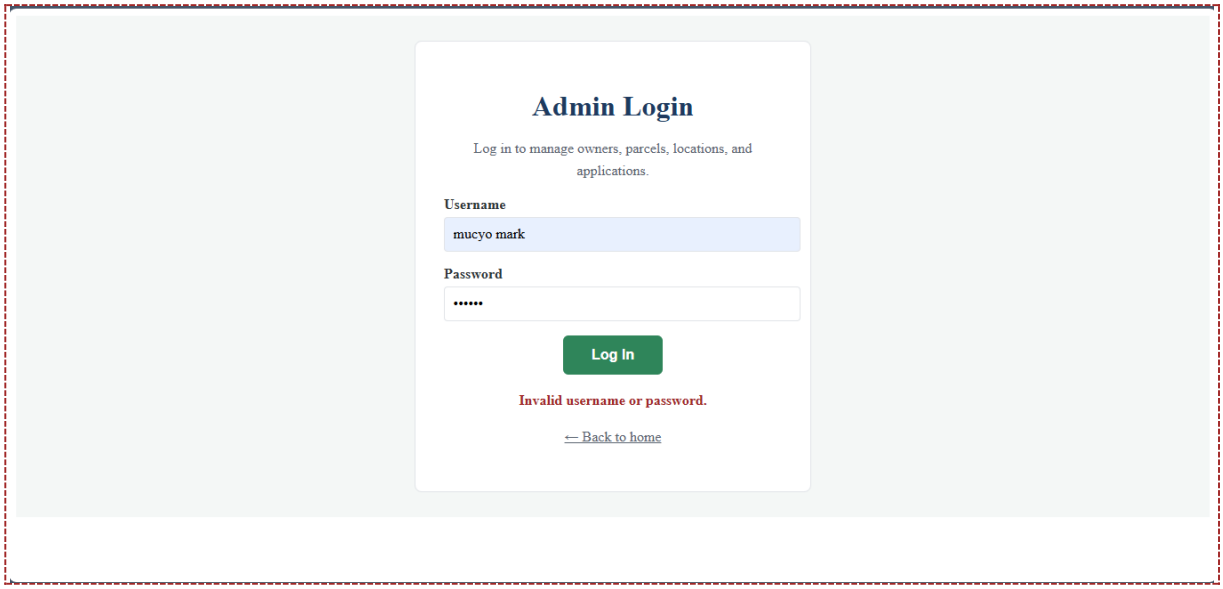
Once the application record is created, the form is hidden and a confirmation box is shown instead, displaying the generated app_id (for example, APP-1752931201234) along with a direct link to the tracking page. This gives the applicant everything they need to check their status later, without requiring any account or login.

Track Application (track.html)



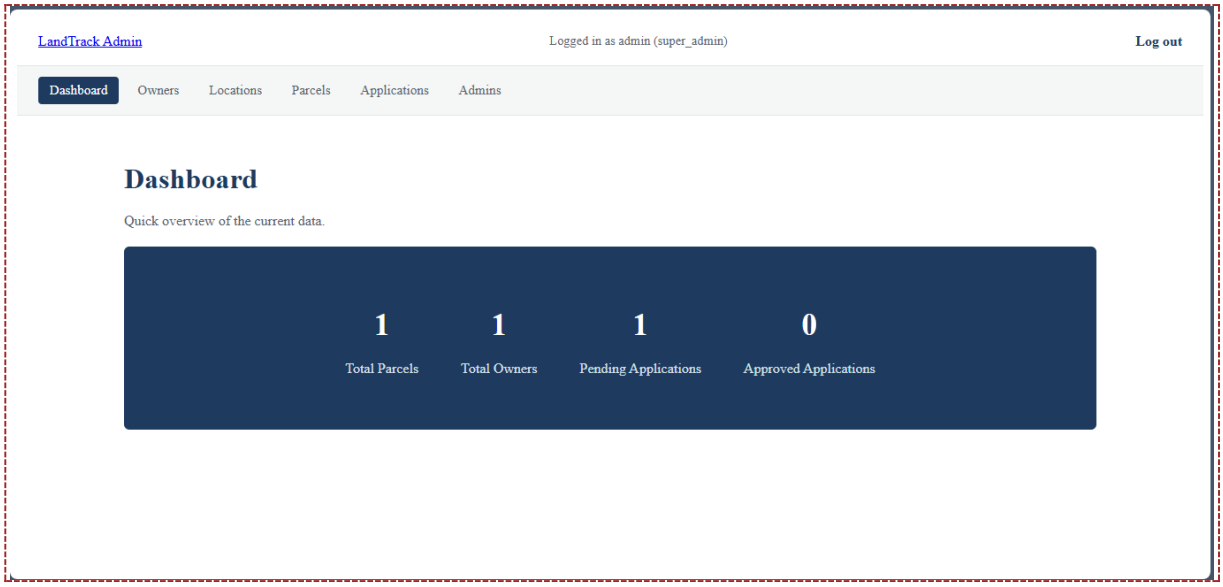
The applicant types their application ID into a single input field. js/track.js searches the in-memory applications array (loaded from data/applications.json) for a matching app_id, then cross-references the linked parcel in data/parcels.json to show its district, cell, land type, and plot number alongside the status. The status is shown as a colored badge — gray for submitted, blue for under review, green for approved, and red for rejected — so it can be read at a glance. If no match is found, a plain message explains that no application was found with that ID.

Admin Login (admin/login.html)



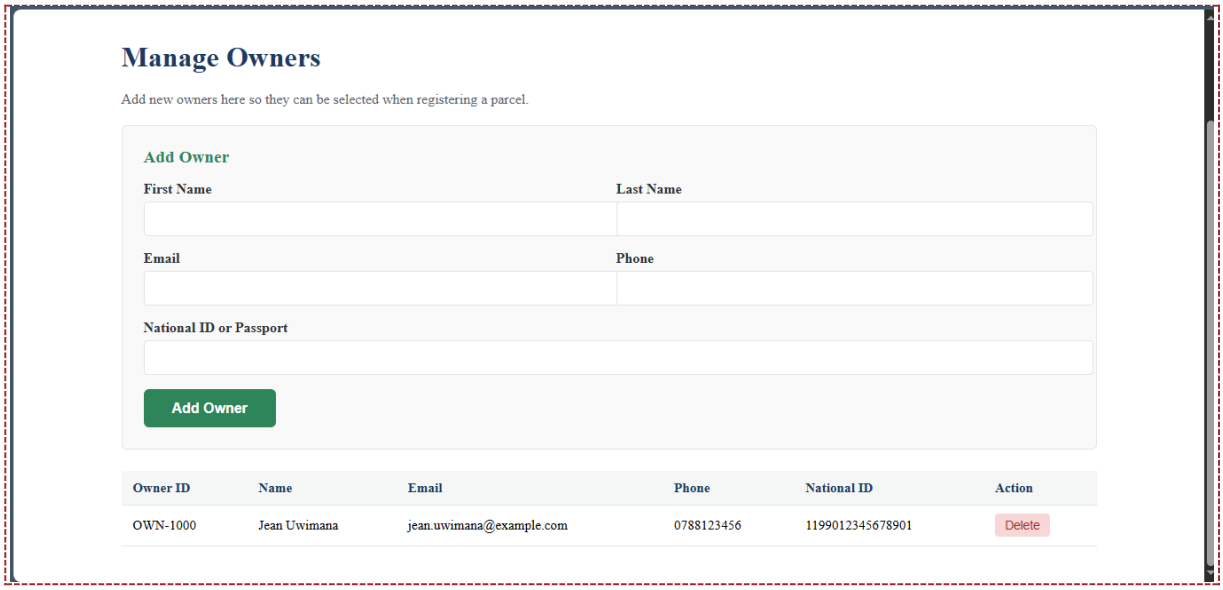
This page loads data/admins.json via AJAX and checks the typed username and password against those records. On a successful match, the browser is redirected to dashboard.html with the admin's username, role, and admin ID attached as a URL query string (for example, dashboard.html?username=admin&role=super_admin&admin_id=ADM-1000). Since the project does not use localStorage or sessionStorage, this query string is how every other admin page recognizes that someone is logged in.

Admin Dashboard (admin/dashboard.html)



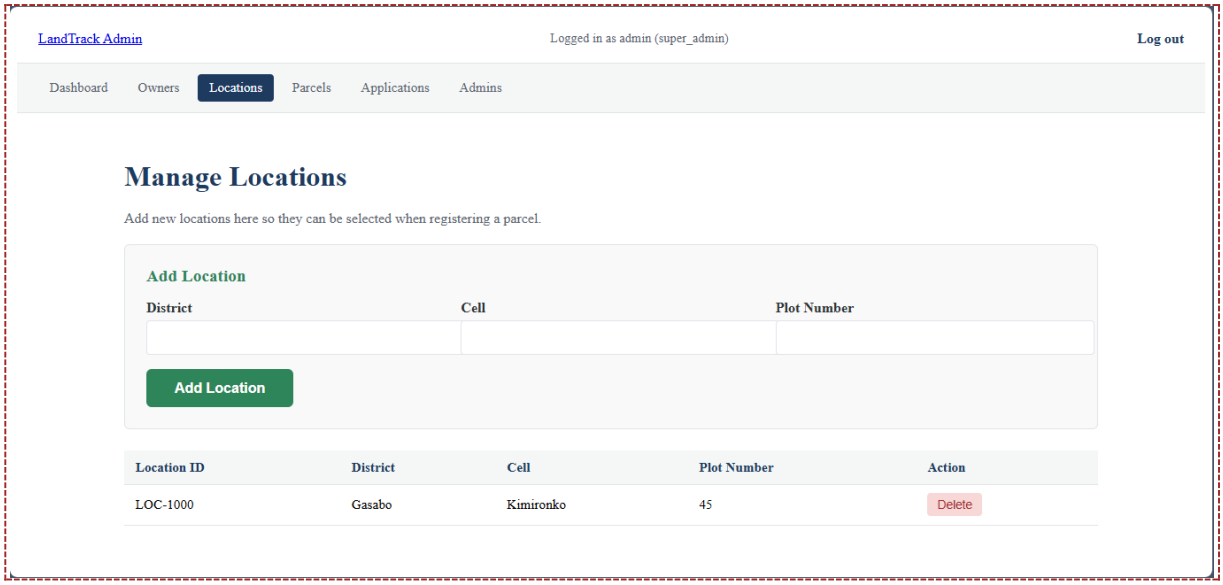
The dashboard reads the login information out of the URL using a small helper function, getQueryParam(), and shows a "Logged in as ..." message. It then loads parcels.json, owners.json, and applications.json via AJAX to calculate four counts: total parcels, total owners, pending applications, and approved applications.

Manage Owners (admin/owners.html)



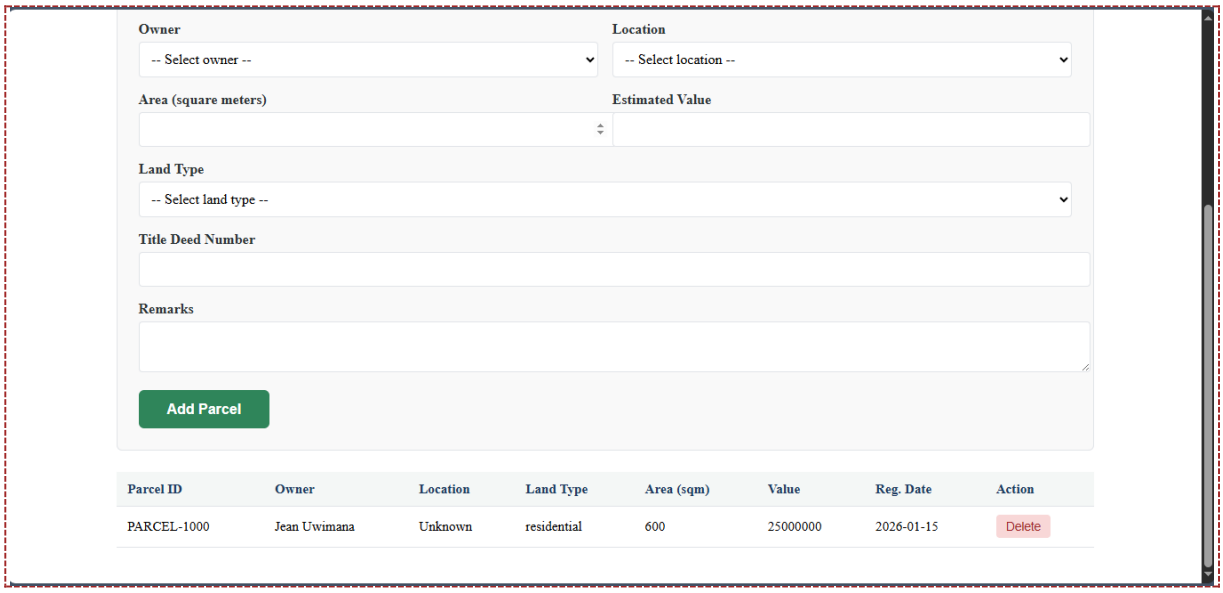
This page lists all existing owner records in a table and provides a form to add a new owner. New owners are pushed into the in-memory owners array and the table re-renders immediately. Each row includes a delete button that removes that owner from the array.

Manage Locations (admin/locations.html)



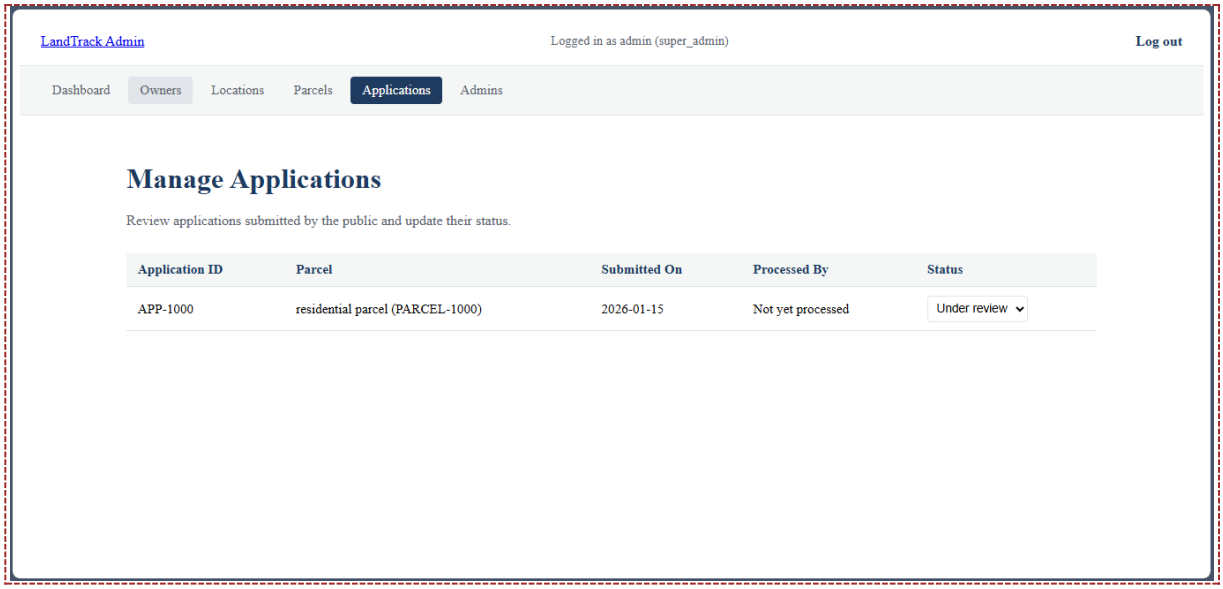
Following the same pattern as the owners page, this page lists existing locations (district, cell, plot number) and lets an admin add new ones, which then become selectable when registering a parcel.

Manage Parcels (admin/parcels.html)



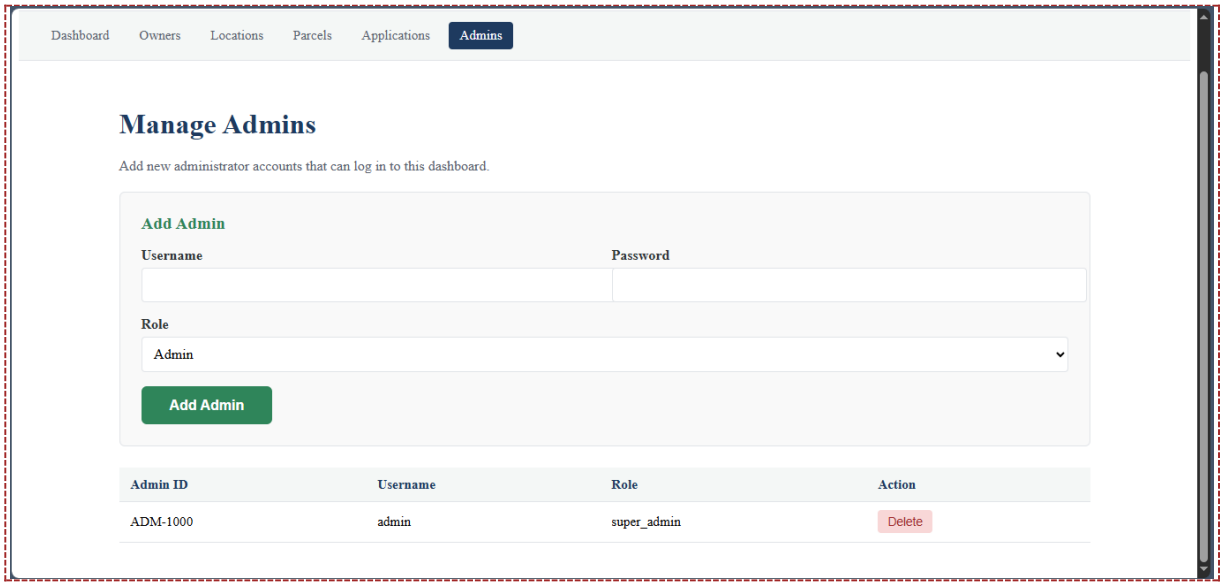
This is the most connected page in the system. It loads owners.json and locations.json to populate two dropdown menus, ensuring a parcel can only be linked to an owner and location that already exist — directly reflecting the foreign-key relationships in the database design. Submitting the form creates a new parcel record that also stores the currently logged-in admin's ID, satisfying the "admin manages parcels" relationship.

Manage Applications (admin/applications.html)



Unlike the other management pages, this page has no add form, since applications can only be created through the public apply.html page. Instead, each row has a status dropdown. Changing it updates that application's status and stamps its processed_by field with the current admin's username, directly implementing the "admin processes applications" relationship from the design.

Manage Admins (admin/admins.html)



This page lists existing admin accounts and lets a logged-in admin add new ones, specifying a username, password, and role (admin or super_admin).

Chapter 4. Conclusion and Recommendation

State of the Implemented System

All pages described in the system design have been implemented: the three public pages (landing, application form, and status tracker) and the seven administrative pages (login, dashboard, owners, locations, parcels, applications, and admin management). The core relationships from the database design — owner owns parcel, location contains parcel, admin manages parcel, parcel has application, and admin processes application — are all represented and enforced through the interface. Based on this, the project can be considered substantially complete relative to its stated objectives. **The only thing missing is backend, some iot**

Recommendations

To the lecturer, about technicalities:

The assignment's constraint of frontend-only technology (no backend, no database engine) necessarily limits data persistence: records created during a session exist only in the browser's memory and are lost on refresh, since browser JavaScript cannot write back to local JSON files. This is a direct, unavoidable consequence of the assignment's own constraints rather than an implementation gap, and is worth keeping in mind when assessing the completeness of the data layer.

To the school, about time constraints:

A project of this scope — covering both a public application flow and a full administrative CRUD zone — benefits from more time allocated to testing across pages, since the interlinked nature of the data (owners, locations, and parcels all referencing each other) means a single page cannot be fully verified in isolation.

To the partners (owners of the business/case study this app targets):

Adopting a system like this in practice would require a real backend and database so that submitted applications and registered parcels persist reliably, along with proper server-side authentication rather than the client-side URL-based approach used here for demonstration purposes.

To the user, about training:

Administrative staff would need a short walkthrough of the dashboard and management pages, particularly the parcel registration workflow, since it depends on owner and location records already existing before a parcel can be linked to them.

To the beneficiaries (applicants/citizens):

The tracking page removes the need to visit an office in person just to check an application's status, provided the application ID is kept safe after submission.

Appendix

Link to the GitHub repository (account and access) of the deployed project:

https://ndikubwimana-didier-gilbert.github.io/Smart_land_mngt_system/